往年题精选：前8讲范围

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

本题库补充五讲正文，收录44题，来自2012—2025年间的期中或期末资料；不含2025第一次阶段测验。按专题挑选有训练价值的题，去掉同型重复、范围外内容以及无法消除歧义的题。它不是全部历年范围内题目的穷尽清单，也不是新增的五天必做量。

默认机器为x86-64/Linux/System V AMD64、AT&T汇编；int32位、long/指针64位、小端、IEEE binary32/binary64。题目明确另给规则时以该题为准。浮点题默认最近偶数舍入、每一步按指定格式舍入，无额外精度或重结合。涉及有符号回绕、超范围转换、union重解释时按题内限定的机器模型；不要把机器结果当作所有C实现都保证的行为。

先独立做题，再看独立解析。原卷链接可能直接包含答案，作答前不要打开。 本题编号Y-A01等保持稳定；“优先”表示推荐挑选，不表示每题都必须做。题目改写后的解释、追问及解析由助手整理，并非原考试官方勘误或评分。

怎么穿插到五讲里

读完哪讲	先挑这些题	有余力再练
第1讲	A02、A06、C04	A01、A04、A07、C01、C05
第2讲	B01、B06、D03	B03、B07、B08、C02、C06、D04
第3讲	D01、E01，并在D06/E05中选一道	D02、D05、E02—E04
第4讲	F04、F05、G01	F01—F03、F06—F07、G02—G04
第5讲纠错	按实际失分选尚未做过的同专题题	2025第一次阶段测验仍单独完整作答

按上表先选12题，其他题按薄弱点选做；已在小班课或主讲义见过同源题时标记“已见”，不作为全新测验。答对结果但说不清理由的题，仍需订正。

A 整数、位操作与字节序

Y-A01 任意位宽的补码与大端存储 · 优先

题源：2017-带答案（来源资料未随仓库分发），一1，PDF文件第2页。

处理：改编：用文字替代原图，补充小端对照。

一个有符号整数类型占 $4N$ 字节，采用补码，$N \geq 1$。把该类型最小负数写入内存。

1. 写出其数值与完整位模式的特征。
2. 大端存储时，最低地址字节是什么？
3. 换成小端存储后，最低、最高地址字节分别是什么？

Y–A02 字符串字节、截断与符号扩展 · 优先

题源：2018–带答案（来源资料未随仓库分发），一2，PDF文件第2页。

处理：改编：改为明确的机器位操作，避开原C指针强转的对齐和别名问题。

低地址到高地址的四个字节是 `32 30 31 38`（十六进制）。在小端机器上：

1. 按32位整数读取，写出十六进制位模式。
2. 算术右移12位，再保留低16位，按16位补码解释，写出位模式与正负。
3. 把这个16位有符号值转换成32位 unsigned，结果是多少？

Y–A03 异或交换逐行追踪 · 选做

题源：2018–带答案（来源资料未随仓库分发），一3，PDF文件第2—3页。

处理：整理：修正原题printf格式，沿用原文件自身的答案修订。

开始时 `x=a, y=b`，依次执行：

```
y = x ^ y;
x = x ^ y;
y = x ^ y;
```

逐行写出 x、y，最后的结果是什么？这里 x、y 是两个独立变量。

Y–A04 从大端字节到小端整数的掩码 · 优先

题源：2019–带答案（来源资料未随仓库分发），一1，PDF文件第2页。

处理：改编：补齐比较括号，原题的运算符优先级会改变含义。

某4字节标识在内存中始终按 `192,168,56,t` 顺序存放，$0 \le t \le 255$。小端机器把这4字节读成32位 unsigned `ip`。填写下面两处十六进制常量：

```
mask = ______;
if ((ip & mask) == ______) { /* 前三个字节匹配 */ }
```

说明为什么不是直接使用 `0xC0A83800`。不需要网络知识。

Y–A05 连续整数的异或 · 选做

题源：2021–带答案（来源资料未随仓库分发），一3，PDF文件第3页。

处理：整理：保留运算与区间，要求说明规律。

```
int s = 0;
for (int i = 0; i < 32; i++) s ^= i;
```

求最终 s，并用分组说明，不必逐项列32行。

Y-A06 负奇数右移与除法 · 优先

题源：2021-带答案（来源资料未随仓库分发），一4，PDF文件第3—4页。

处理：整理：明确负数采用算术右移。

32位 int，负数算术右移：

```
int x = 0x2021;
int y = -x;
int z = (x/2) - (x>>1) + (y/2) - (y>>1);
```

求四个除法/移位结果以及 z。

Y-A07 比较前先确定转换后的类型 · 优先

题源：2016期末-带答案（来源资料未随仓库分发），一1，PDF文件第2页。

处理：整理：逐项判断，明确超范围unsigned转int使用课程补码模型。

int 和 unsigned 均为32位；本题中 `(int)2147483648U` 按位模式解释为 INT_MIN。判断各式真假，并说明比较类型：

```
0 == 0U
2147483647U > -2147483647-1
(unsigned)-1 < -2
2147483647 > (int)2147483648U
```

Y-A08 lowbit命题遗漏了什么 · 选做

题源：2023-带答案（来源资料未随仓库分发），一2D、一10，PDF文件第2—3、6页。

处理：改编：用unsigned明确模运算，并把原题漏洞变为边界检查。

定义 `unsigned lowbit(unsigned x) { return x & (0u-x); }`，unsigned为32位。

1. 求 x=12、x=0、`x=0x80000000u` 时的结果。
2. “对任意x，结果都等于某个 2^k”是否成立？应怎样补条件？

B 浮点编码、舍入与范围

Y-B01 没有符号位的小浮点格式 · 优先

题源：2012-带答案（来源资料未随仓库分发），Problem B第2题，PDF文件第4页。

处理：整理翻译：保留参数、数值和round-to-even规则。

一种5位浮点格式没有符号位：exp占3位、frac占2位，Bias=3。exp=000为非规格化，111保留作特殊值，其余按IEEE式规格化处理。采用最近偶数舍入。 分别把 $3, 9, 3/16, 15/2$ 编码为 `exp frac`，写出舍入后的实际数值；说明两次中点舍入为什么选那个邻居。

Y-B02 最小正规格化数为什么不含n · 选做

题源：2017-带答案（来源资料未随仓库分发），一3，PDF文件第2—3页。

处理：整理：保留一般参数，增加最小非规格化对照。

IEEE式格式有1位符号、k位exp、n位frac，Bias=$2^{k-1}-1$。 写出最小正规格化数和最小正非规格化数，解释两者为什么相差 2^n 倍。

Y-B03 最大间距和半个ULP · 选做

题源：2015-带答案（来源资料未随仓库分发），一1，PDF文件第2页。

处理：改编：明确最近舍入及有限可表示范围，避免无界实数带来的歧义。

binary32有23位frac。对绝对值不超过最大有限binary32值的实数，采用最近偶数舍入。

1. 最大指数区间 $[2^{127}, 2^{128})$ 内，相邻有限数间距是多少?
2. 本题限定范围内，舍入的最大绝对误差是多少?

Y-B04 标准单精度编码 · 优先

题源：2015期末-20160104-带答案（来源资料未随仓库分发），一2，PDF文件第2页。

处理：整理：保留27.5，增加字段分解。

把27.5编码为binary32，分别给出符号位、exp、frac及32位十六进制位模式。

Y-B05 浮点数加一什么时候失效 · 优先

题源：2019-带答案（来源资料未随仓库分发），一3，PDF文件第3页。

处理：整理：明确每步binary32、最近偶数舍入，不采用额外精度或重结合。

```
for (float f = 1; ; f = f + 1) {
    if (f + 1 - f != (float)1) {
        printf("%.0f\n", f);
        break;
    }
}
```

程序会输出什么？请分别解释到达该值以前、到达该值时的行为。

Y-B06 连续五个整数的舍入误差 · 优先

题源：2020-带答案（来源资料未随仓库分发），二，第二段代码，PDF文件第7页。

处理：整理：保留原值，明确最近偶数舍入。

```
int a = 33554442;  // 2^25+10
int b = a + 5;
for (; a < b; a++) {
    float f = a;
    printf("%d ", (int)f - a);
}
```

写出五次输出，并为每个a列出f的实际整数值。所有int/float分别为32位整数和binary32。

Y-B07 总位数不变，有限实数数量会不会变 · 选做

题源：2024期中-无答案（来源资料未随仓库分发），一3，PDF文件第2页。

处理：整理：明确正负零只算同一个实数。

标准32位IEEE式格式有1位符号、8位exp、23位frac。现把exp改为10位、frac改为21位，仍保留全1阶码作Inf/NaN。 可表示的不同有限实数数量是更多、更少还是相同？推导一般公式，不只讨论最大数或精度。

Y-B08 给定新规则后重新推导FP8 · 选做

题源：2023-带答案（来源资料未随仓库分发），二1—3，PDF文件第13—14页。

处理：截取整理：只保留格式推导，移除神经网络选型部分。

格式甲 E5M2：1位符号、5位exp、2位frac，Bias=15，沿用IEEE式特殊值规则。 格式乙 E4M3：1位符号、4位exp、3位frac，Bias=7；exp=0000仍按非规格化处理，exp=1111也可表示规格化有限数，只有exp=1111且frac=111时为NaN，不设Inf。

1. 给出甲的最大正有限数、最大正非规格化数的编码与数值。
2. 给出乙的最大正有限数的编码与数值。若乙沿用标准特殊值规则，最大值原本是多少?
3. 把有符号int8的全部数值转为乙：会不会溢出？会不会发生舍入？举例。

C 汇编指令、寄存器与条件码

Y-C01 用lea组合乘法 · 选做

题源：2013-带答案（来源资料未随仓库分发），一5，PDF文件第2页。

处理：整理：保留原汇编，补充按低32位观察结果。

参数n在rdi中，函数返回int，观察返回值低32位：

```
leaq (%rdi,%rdi,4), %rdi
leal (%rdi,%rdi,1), %eax
ret
```

对应 `return n * ____;`，填常数并逐步说明。

Y-C02 哪些指令会改CF · 优先

题源：2016-带答案（来源资料未随仓库分发），一1，PDF文件第2页。

处理：整理：补齐指令后缀。

下列指令中哪条会改变CF？说明其余为什么不改。

```
jmp NEXT
jc NEXT
incw %bx
shlw $1, %ax
```

Y-C03 test会把寄存器清零吗 · 选做

题源：2016-带答案（来源资料未随仓库分发），一5，PDF文件第2页。

处理：整理：保持指令含义，改为逐项解释。

比较下列四条指令各自执行后的AX：

```
subw %ax, %ax
xorw %ax, %ax
testw %ax, %ax
andw $0, %ax
```

哪些可以无条件把AX清零?

Y-C04 写32位寄存器和符号扩展 · 优先

题源：2018-带答案（来源资料未随仓库分发），一4，PDF文件第3页。

处理：整理：保留四个知识判断并纠正描述。

判断并说明：

1. movl以寄存器为目的时，会清零其高32位。
2. cltq把eax符号扩展为rax。
3. `movabsq $Imm, D` 的完整64位立即数形式允许D为任意内存操作数。
4. movswq把16位字零扩展成32位双字。

Y-C05 语法合法不等于程序合理 · 优先

题源：2021期末-无答案（来源资料未随仓库分发），一2，PDF文件第3页。

处理：整理：保持选项，增加实际效果解释。

哪些是合法x86-64指令?

```
A: movq %rax, $1
B: xorq %rsp, %rsp
C: movq (%rax), (%rbx)
D: movq (%rax,%rbx,3), %rcx
```

合法的指令若放在普通函数中执行，有什么风险?

Y-C06 32位加法同时考寄存器和标志 · 优先

题源：2025期末-无答案（来源资料未随仓库分发），一5，PDF文件第3页。

处理：整理：保留数值，直接询问最终状态。

初始 `rax=0x12345678FFFFFFFF`，`rcx=1`，执行：

```
addl %ecx, %eax
```

求完整rax及CF、ZF、SF、OF。

D 条件、循环与跳转表

Y-D01 相对跳转从哪里开始加 · 优先

题源：2016-带答案（来源资料未随仓库分发），一3，PDF文件第2页。

处理：整理：保留原地址与机器码。

```
400020: 74 f0    je TARGET
400022: 5d       pop %rbp
```

若跳转发生，TARGET是多少？说明位移f0的符号和计算基准。

Y-D02 跳转表先做了加一 · 优先

题源：2016-带答案（来源资料未随仓库分发），一6，PDF文件第3页。

处理：整理：保留表项，询问映射而不武断推断源代码case标签是否存在。

入口rdi为原始switch值i：

```
addq $1, %rdi
cmpq $8, %rdi
ja .L2
jmp *.L4(,%rdi,8)
.L4:
.quad .L9, .L5, .L6, .L7, .L2, .L7, .L8, .L2, .L5
```

1. 哪些i进入表中?
2. 列出每项对应的i与目标。
3. 哪两个表内i直接跳到默认路径.L2？为什么不能仅凭此断言源代码没有这两个case标签?

Y-D03 从cmov还原三目运算 · 优先

题源：2014期末-带答案（来源资料未随仓库分发），一3，PDF文件第2—3页。

处理：整理：保留代码，明确只考虑无有符号溢出的输入。

参数int x在edi、int y在esi；以下32位加减的数学结果均可表示：

```
leal (%rsi,%rdi), %eax
movl %esi, %edx
subl %edi, %edx
cmpl %esi, %edi
cmovge %edx, %eax
ret
```

补全 `return (x < y) ? ____ : ____;`，解释cmp和cmov的对应关系。

Y-D04 不能用条件传送掩盖非法访存 · 优先

题源：2024期中-无答案（来源资料未随仓库分发），一8，PDF文件第4页。

处理：改编：不要求编译器必须输出同一代码，明确比较语义。

```
long read1(long *p) { return p ? *p : 0; }
long read2(long *p) { if (p) return *p; else return 0; }
```

p为NULL或指向可读long的有效指针。

1. 两函数语义是否相同?
2. 能否先无条件读出*p，再用cmov选择结果?
3. 如果cmov的源是内存，条件为假就一定不会访问该内存吗?

Y-D05 把循环变量映射到栈槽 · 选做

题源：2013-带答案（来源资料未随仓库分发），三，PDF文件第6—7页。

处理：截取改编：保留平方根算法的一轮，省略算法证明与大段机器码。

原程序一轮为：

```
b = y + m;
y >>= 1;
if (x >= b) { x -= b; y += m; }
m >>= 2;
```

x、y、m均为unsigned long。栈槽对应：x在-0x28(rbp)，b在-0x18(rbp)，m在-0x10(rbp)，y在-0x8(rbp)。

```
movq -0x10(%rbp), %rax
movq -0x8(%rbp), %rdx
leaq ______, %rax          # b=y+m
movq %rax, -0x18(%rbp)
shrq -0x8(%rbp)
movq -0x28(%rbp), %rax
cmpq -0x18(%rbp), %rax
______ .Lskip              # x<b时跳过更新
movq -0x18(%rbp), %rax
subq %rax, -0x28(%rbp)
movq -0x10(%rbp), %rax
addq %rax, -0x8(%rbp)
.Lskip:
shrq $______, -0x10(%rbp)
```

补三处空。另从x=10、y=0、m=4开始，追踪到m=0的各轮（这是为练习缩小的初始状态，不是原程序通用初始化）。

Y-D06 完整switch：跳转表、落入、除法偏置 · 优先

题源：2016-带答案（来源资料未随仓库分发），三(1)，PDF文件第9—10页。

处理：截取整理：保留完整switch与地址，增加两个检验输入。

```
int foo(int x, int i) {
    switch (i) {
    case 1: x -= 10;
    case 2: x *= 8; break;
    case 3: x += 5;
    case 5: x /= 2; break;
    case 0: x &= 1;
    default: x += i;
    }
    return x;
}
```

输入限定为C中的加减乘不溢出。对应反汇编如下；移位按给定机器指令分析：

```
4004a8: mov  %edi,%edx
4004aa: cmp  $0x5,%esi
4004ad: ja   4004d4
4004af: mov  %esi,%eax
4004b1: jmpq *0x400690(,%rax,8)
4004b8: sub  $0xa,%edx
4004bb: shl  $0x3,%edx
4004be: jmp  4004d6
4004c0: add  $0x5,%edx
4004c3: mov  %edx,%eax
4004c5: shr  $0x1f,%eax
4004c8: lea  (%rdx,%rax),%eax
4004cb: mov  %eax,%edx
4004cd: sar  %edx
4004cf: jmp  4004d6
4004d1: and  $0x1,%edx
4004d4: add  %esi,%edx
4004d6: mov  %edx,%eax
4004d8: retq
```

1. 填出跳转表6个地址（对应i=0到5）。
2. 哪些分支发生落入？i=4走哪里？
3. 解释 `shr $31`、lea、sar为何实现向零除以2。
4. 求foo(13,1)、foo(-8,3)。

E 过程调用与递归

Y-E01 第七个参数前还有返回地址 · 优先

题源：2019-带答案（来源资料未随仓库分发），一5，PDF文件第4页。

处理：整理：限定System V、全部参数为long，避免混合类型ABI歧义。

System V AMD64下，函数所有参数都是long，至少7个。刚执行完call、函数入口还未修改rsp，令rsp=S。 第k个参数（k>6）存放在哪里？S处存放什么？列出第7、第8个参数的地址。

Y-E02 栈帧、对齐与leave的准确含义 · 优先

题源：2025期末-无答案（来源资料未随仓库分发），一6，PDF文件第3页。

处理：改编：补充省略区间是否改变rsp的条件，避免leave等价式歧义。

进入baz时rsp=S，且S模16余8：

```
pushq %rbp
movq %rsp, %rbp
subq $48, %rsp
# 此后直到call前，rsp不再变化
callq qux
# qux正常返回，之后rsp不再变化
leave
ret
```

1. 保存旧rbp、返回地址、48字节局部区分别在哪里？
2. call qux前rsp模16是多少？
3. 在这里leave是否等价于 `addq $48,%rsp; popq %rbp`？一般情况下leave的准确等价操作是什么？

Y-E03 八参数调用的压栈顺序 · 选做

题源：2025期末-无答案（来源资料未随仓库分发），一7，PDF文件第3—4页。

处理：截取整理：省略前六个寄存器赋值，补充栈对齐前提。

前六个long参数已放入规定寄存器，下面指令执行前rsp按16字节对齐：

```
pushq a8
pushq a7
callq f
addq $16, %rsp
```

1. 在f入口，a7、a8相对rsp的偏移分别是多少？
2. 如果交换两条push的顺序会怎样？
3. 谁回收传参空间？能否擅自改用r10、r11传第7、8参？f必须建立rbp帧指针吗？

Y-E04 call *rax和call *(rax)差一层 · 选做

题源：2017-带答案（来源资料未随仓库分发），一6，PDF文件第3页。

处理：改编：明确rax中已保存函数地址，不讨论编译器是否消除间接调用。

rax中已经是目标函数的入口地址。下列哪条正确？其他各项有什么不同？

```
call *%rax
call (%rax)
call *(%rax)
```

Y-E05 递归追踪与每层栈帧 · 优先

题源：2021-无答案（来源资料未随仓库分发），三，PDF文件第10—12页。

处理：截取改编：保留递归逻辑和帧大小，改为明确的状态表与深度问题。

由原题汇编整理出以下函数，限定n、m为非负且运算不溢出：

```
long f(long n, long m) {
    if (n == 0 || m == 1) return m;
    if (m & 1) return f(n - 1, 3*m + 1);
    return f(n - 1, m >> 1);
}
```

每一层实际生成的汇编均有 `pushq %rbp; movq %rsp,%rbp; subq $0x20,%rsp`，并使用普通call递归，不做尾调用优化。

1. 列出f(7,6)各层入口的(n,m)，求返回值。
2. 最深处共有多少个同时活动的f调用?
3. 相邻两层刚执行完各自上述序言时，rsp相差多少字节？分项说明。
4. 若另一个编译器做了尾调用优化，能否仍断言每次递归都增加一帧?

F 数组、结构体与联合

Y-F01 同一结构体放到不同ABI下 · 选做

题源：2015-带答案（来源资料未随仓库分发），一5，PDF文件第3页。

处理：改编：保留字段，增加x86-64对照并显式给出两套对齐规则。

```
struct S {
    char a; double *b; double c;
    short d; long long e; short f;
};
```

环境甲：IA32，指针4字节，double和long long各8字节但按4字节对齐；char/short分别按1/2字节对齐。 环境乙：本课x86-64，指针8字节，指针/double/long long按8字节对齐；char/short仍按1/2字节对齐。 分别给出原顺序的成员偏移、sizeof，并给出重排后可达到的最小sizeof及一种顺序。

Y-F02 union没有被写过的字节 · 选做

题源：2015-带答案（来源资料未随仓库分发），一7，PDF文件第4页。

处理：改编：明确int为4字节且其余字节未指定。

```
union U { unsigned char c[2]; unsigned int i; };
```

unsigned int为4字节。大端机器上只知道共享存储的前两个字节为01、23，另两个字节没有给定。 能否断定整数视图为0x01230000？若希望唯一算出结果，还缺什么信息?

Y-F03 double和char解释共享字节 · 优先

题源：2018-带答案（来源资料未随仓库分发），一5，PDF文件第3—4页。

处理：改编：补齐大小端、char符号性和布局前提。

```
union U {
    double d;
    struct { int i; char c[4]; } s;
};
```

int为4字节，c从偏移4开始，double采用IEEE binary64。已知d=1.0的位模式为 `0x3FF0000000000000`，按本课机器的union字节重解释模型：

1. 小端、char为signed char时，s.c[2]提升到int是多少?
2. 小端、char为unsigned char时是多少?
3. 大端时该字节又是什么?

Y-F04 结构体偏移直接出现在指令里 · 优先

题源：2018期末-带答案（来源资料未随仓库分发），一2，PDF文件第2—3页。

处理：整理：保留结构体和汇编空格，明确System V x86-64。

```
struct A { char c; int i; double d; int array[10]; };
struct B { int array[10]; double d; char c; int i; };
void foo(struct A *pa, struct B *pb, int index) {
    pb->i = pa->array[index];
}
```

index在0到9之间。补全：

```
movslq %edx, %rdx
movl ___(%rdi,%rdx,___), %eax
movl %eax, ___(%rsi)
```

并给出sizeof(A)、sizeof(B)。

Y-F05 从地址步长反推二维数组 · 优先

题源：2021-带答案（来源资料未随仓库分发），一7，PDF文件第5页。

处理：改编：求B、C及A的下界，避免原选择题将A错误理解为唯一可推导。

```
int a[A][B];
for (int i=0; i<C; i++) a[i][C-i]=1;
```

初始rdi为a的首地址：

```
leaq 40(%rdi), %rax
addq $440, %rdi
.L2:
movl $1, (%rax)
addq $40, %rax
cmpq %rdi, %rax
jne .L2
```

假设int占4字节且所有数组访问合法。求B、C；A能唯一确定吗？至少是多少？

Y-F06 逐层解读数组指针的sizeof · 选做

题源：2023-带答案（来源资料未随仓库分发），一9，PDF文件第6页。

处理：整理：保留原声明。

System V x86-64，int4字节、指针8字节：

```
int (*A[2])[3];
```

分别求 `sizeof A`、`sizeof *A`、`sizeof **A`、`sizeof ***A`，再求总和。每步先说明类型。

Y-F07 结构体加多层声明混合题 · 选做

题源：2024期中-无答案（来源资料未随仓库分发），一4，PDF文件第3页。

处理：整理：改为求表达式的值，避免原printf格式与ptrdiff_t不匹配。

```
struct student {
    int id; char name[6]; short age;
    char gender; float score;
};
struct student s;
struct student *(*p[8])[4];
```

System V x86-64常见对齐规则。求：

1. `sizeof(s)`；
2. `sizeof(**p)`；
3. `(char *)&p[2] - (char *)p`；
4. `&p[4] - p`； 最后求四个结果之和。

G 内存安全与防护

Y-G01 把缓冲区加大是否就安全 · 优先

题源：2016期末-带答案（来源资料未随仓库分发），一3，PDF文件第3页。

处理：整理：保留方案，要求区分作用。

以下四种措施，哪一项本身不能提供可靠的越界保护？其余各项分别解决什么问题？ A. 栈中放置canary，并在返回前检查。 B. 使用正确限制长度的输入/拷贝操作。 C. 随机化栈地址。 D. 尽可能把缓冲区数组开大。

Y-G02 canary应检查相等还是大小 · 选做

题源：2024期中-无答案（来源资料未随仓库分发），一6C，PDF文件第3—4页。

处理：截取改编：只保留canary条件判断，排除其他选项的绝对化描述。

某答案声称：“返回前把栈中canary与原值cmp，然后用jle判断是否可以正常返回。”这是否合适？写出应该检查的关系，以及可采用的条件跳转。

Y-G03 读懂真实栈保护代码 · 选做

题源：2019-带答案（来源资料未随仓库分发），三中caller的canary片段，PDF文件第12—14页。

处理：截取整理：只取栈保护逻辑，指定外部保存值含义，不考TLS细节。

本题把 `%fs:0x28` 视为系统保存的原canary值，不要求理解段寄存器。函数序言后有：

```
movq %fs:0x28, %rax
movq %rax, -0x8(%rbp)
```

返回前有：

```
movq -0x8(%rbp), %rax
xorq %fs:0x28, %rax
je .Lok
callq __stack_chk_fail
.Lok:
leave
ret
```

说明canary保存位置，正常路径中rax与ZF的值，以及哪种情况下进入失败处理。单有这种检查能保证所有局部变量都没被修改吗?

Y-G04 变长栈帧不是越界防护 · 选做

题源：2022-无答案（来源资料未随仓库分发），一10，PDF文件第4页。

处理：整理：保留四种技术并明确只问设计目的。

canary、变长栈帧、不可执行栈、ASLR中，哪一项本身不是为了防止缓冲区溢出攻击而设计的？再说明不可执行栈为何不能单独阻止使用已有代码片段的返回链。